Enterprise-to-Agentic (E2A)

Architecture Framework

Multi-Cloud AI Ecosystem Reference — AWS · GCP · Azure · Meta Llama

Framework repository: `github.com/subhamviky/e2a-framework`
Document role: Cloud ecosystem reference for the E2A Framework — maps abstract class contracts to AWS, GCP, Azure, and Meta Llama managed AI services
Companion documents: Master Abstraction Reference · Implementation Playbook

How This Relates to Other E2A Documents
The Master Abstraction Reference defines the class contracts (WHAT). The Implementation Playbook provides the e2a_base.py scaffold and usage guide (HOW). This document maps those contracts to cloud managed services (WHERE) and provides a decision framework for vendor selection (WHEN). Read all three in sequence.

1. Purpose & Structure

This document answers two questions for every vendor: (1) which managed service implements which E2A abstract class, and (2) how to write a correct, E2A-compliant concrete implementation using that vendor's SDK. A consistent 6-pillar structure is used across all vendors, followed by an 11-step execution mapping, corrected code examples, and a cross-cloud decision framework.

The 6-pillar structure used throughout:

- Pillar 1 — Responsible AI: security, governance, guardrails, data protection
- Pillar 2 — Data Management: vector stores, RAG pipelines, caching, data lakes
- Pillar 3 — Model Hub: foundation model access and hosting
- Pillar 4 — Orchestration: agent orchestration, workflow execution, tool calling
- Pillar 5 — Observability: metrics, tracing, logs, SLO monitoring
- Pillar 6 — GenAI Ops: CI/CD, evaluation, model training, IaC

***Attribution**: The E2A Framework's abstract classes were independently identified to mirror enterprise distributed systems. Their current organization into six pillars (Responsible AI, Data Management, Model Hub, Orchestration, Observability, and GenAI Ops) is a structural mapping inspired by the AWS AI Ecosystem visualization by Prashant Rathi.*

Access Modifier Compliance Throughout
All code in this document strictly follows the E2A access modifier contract: run()/execute()/retrieve() PUBLIC; _build_messages/_apply_policy/_evaluate_output/_fallback/_handle_error PROTECTED (single underscore); __admit_quota/__http_post PRIVATE (double underscore); BaseLLMAgentService is the one class with two public entry points, process_inference() and process_inference_stream(). Earlier drafts had violations in Azure, GCP, and Llama examples — all corrected here.

2. Core Portability Principle

The E2A Framework's vendor independence comes from one design decision: the model call is not in the agent at all. Every agent reaches a model through config['llm_service'] — a BaseLLMAgentService — and a provider is one concrete subclass of it that implements two transport hooks, _dispatch_upstream_call() and _dispatch_upstream_stream(). Quota, concurrency, context budgeting, caching, DLP and token accounting are inherited and identical on every provider. The SAME agent subclass runs on any cloud by binding a different service; switching model within a provider is a single config value.

One subclass — four clouds — zero code changes

```
# Same OrderOpsAgent runs on any provider — only the bound LLM service changes:

# AWS Bedrock — Claude 3.5 Sonnet
result = agent.run(state, {'llm_service': bedrock_svc, 'model_id': 'anthropic.claude-3-5-sonnet-
20241022-v2:0'})
# GCP Vertex AI — Gemini 2.5 Pro
result = agent.run(state, {'llm_service': vertex_svc, 'model_id': 'gemini-2.5-pro'})
# Azure AI Foundry — GPT-4o
result = agent.run(state, {'llm_service': azure_svc, 'model_id': 'gpt-4o'})
# Meta Llama 4 via Bedrock — same service class, different model_id
result = agent.run(state, {'llm_service': bedrock_svc, 'model_id': 'meta.llama4-maverick-17b-
instruct-v1:0'})
# Meta Llama 4 standalone (on-prem / local)
result = agent.run(state, {'llm_service': ollama_svc, 'model_id': 'llama4-scout-17b'})

# BaseLLMAgentService (in e2a_base.py) owns quota, budgeting, caching, DLP, accounting.
# A provider is one subclass — nothing else in the stack is provider-specific:
class BedrockLLMService(BaseLLMAgentService):
    async def _dispatch_upstream_call(self, messages, model_id, temperature,
            max_tokens, tools, stop_sequences, cache_handle, config, **kw): ...
    async def _dispatch_upstream_stream(self, messages, model_id, temperature,
            max_tokens, config, **kw): ...
class VertexLLMService(BaseLLMAgentService): ...        # same two hooks
class AzureOpenAILLMService(BaseLLMAgentService): ...   # same two hooks
class OllamaLLMService(BaseLLMAgentService): ...        # same two hooks (standalone / on-prem)
# Adding a provider = adding a subclass. No agent, no scaffold line changes.
# Substring routing on model_id is gone: the binding, not the string, selects the provider.
```

2.1 Provider Transport Subclasses

The body of each provider's _dispatch_upstream_call(). These four are the only places in the framework where a vendor SDK is invoked.

```
# 2.1 sketches — illustrative; verify SDK versions before use.
# Each hook returns {'text', 'metadata': {'tokens_used'}} and may add 'tool_calls'.

# --- AWS: Amazon Bedrock Converse
async def _dispatch_upstream_call(self, messages, model_id, temperature, max_tokens,
                                  tools, stop_sequences, cache_handle, config, **kw):
    system = [{'text': m['content']} for m in messages if m['role'] == 'system']
    turns = [{'role': m['role'], 'content': [{'text': m['content']}]}
             for m in messages if m['role'] != 'system']
    r = await asyncio.to_thread(self._client.converse, modelId=model_id,
          system=system, messages=turns,
          inferenceConfig={'maxTokens': max_tokens, 'temperature': temperature})
    return {'text': r['output']['message']['content'][0]['text'],
            'metadata': {'tokens_used': r['usage']['totalTokens']}}

# --- GCP: Vertex AI (Gemini)
    r = await GenerativeModel(model_id).generate_content_async(
          self._to_contents(messages),
```

```
            generation_config={'temperature': temperature, 'max_output_tokens': max_tokens})
    return {'text': r.text,
            'metadata': {'tokens_used': r.usage_metadata.total_token_count}}

# --- Azure: Azure OpenAI Chat Completions
    r = await self._client.chat.completions.create(model=model_id, messages=messages,
          temperature=temperature, max_tokens=max_tokens)
    return {'text': r.choices[0].message.content,
            'metadata': {'tokens_used': r.usage.total_tokens}}

# --- Standalone: Ollama
    r = await self._http.post(config['endpoint'], json={'model': model_id,
          'messages': messages, 'stream': False})
    j = r.json()
    return {'text': j['message']['content'],
            'metadata': {'tokens_used': j.get('prompt_eval_count', 0) + j.get('eval_count', 0)}}
```

3. Amazon Web Services (AWS)

Bedrock · SageMaker · OpenSearch · CloudWatch · X-Ray

3.1 AWS AI Ecosystem — 6-Pillar Breakdown

E2A Pillar	Managed Service	E2A Component Mapped
1. Responsible AI	AWS IAM + Identity Center, Bedrock Guardrails, KMS, Macie, Comprehend, SageMaker Model Registry	BaseGovernanceFramework._apply_policy() → Bedrock Guardrails (denied topics, PII redaction); IAM enforces approved_agents list
2. Data Management	OpenSearch Service (BM25+KNN hybrid), Aurora pgvector, MemoryDB, Bedrock Knowledge Bases, S3, Lake Formation	BaseRAGPipeline._search_index(query_text, query_vector) → OpenSearch hybrid (text → BM25 leg, vector → KNN leg); _embed_text() → Titan Embeddings via Bedrock; _index_document() → S3+KB
3. Model Hub	Amazon Bedrock: Nova, Claude 3.5/4, Llama 4, Mistral, Cohere; SageMaker JumpStart for fine-tuned models	BedrockLLMService._dispatch_upstream_call() → Bedrock Converse API; model swap via config['model_id'], no agent change. Bedrock Service Quotas (RPM/TPM) → max_rpm / max_tpm
4. Orchestration	Bedrock Agents, Bedrock Flows, Step Functions, LangGraph on Lambda/ECS	BaseWorkflow._build_workflow() → LangGraph StateGraph; _get_agent() → agent registry; compare with Bedrock Flows §3.3
5. Observability	CloudWatch Logs+Alarms, AWS X-Ray, CloudTrail, SageMaker Model Monitor, Container Insights	BaseObservability._collect_metrics() → CloudWatch PutMetricData; _collect_traces() → X-Ray; _enforce_slo() → CW Alarms
6. GenAI Ops	Bedrock Evaluation, SageMaker MLflow, CDK/CloudFormation, CodePipeline, ECR, ECS Fargate	BasePipeline._run_rag_eval() → Bedrock Evaluation; _build_artifacts() → ECR; _deploy_service() → ECS rolling deploy

Source Credit: *Service mappings in this section align with the AWS AI Ecosystem Reference (Prashant Rathi, 2024-2026). E2A components represent independent implementation logic mapped onto these cloud-native primitives.*

Managed Service Disclaimer*: Certain ecosystem components referenced in this mapping (e.g., Pinecone, LangGraph, Lakera) are third-party partner technologies and are not native managed services of AWS, Google Cloud, or Microsoft Azure.*

3.2 E2A to AWS — 11-Step Execution Mapping

#	E2A Step	Managed Tool Equivalent	Enterprise Implication

1	`BaseValidationService.validate()` `(before run())`	Bedrock Agent input parsing + API Gateway request validation	Prevents malformed payloads consuming LLM tokens. API Gateway enforces the input contract at the edge.
2	`Governance check`	AWS IAM + Identity Center (agent identity)	Least-privilege. Only authorized principals trigger specific intents.
3	`_apply_policy`	Bedrock Guardrails pre-processing (denied topics, PII redaction)	Safety before data reaches model weights. Non-developers update policies in AWS Console without code changes.
4	`__ensure_idempotency`	DynamoDB conditional writes + ClientToken	Prevents duplicate side-effects (e.g., charging twice) across distributed retries.
5	`_build_messages`	Bedrock Prompt Management (versioned templates)	Decouples system instructions from code. Enables Prompt Ops — A/B testing without redeployment.
6	`__track_tokens`	Bedrock Inference Profiles + CloudWatch token metrics	FinOps control. Granular cost-per-intent visibility. Automated throttling on budget breach.
7	`LLM service — _dispatch_upstream_call`	BedrockLLMService → Bedrock Converse API (Nova/Claude/Llama/Mistral unified)	Provider abstraction in one subclass. Swap model family via config['model_id']; swap provider by binding another service. Quota, concurrency and DLP inherited from BaseLLMAgentService.
8	`_evaluate_output`	Bedrock Guardrails post-processing (contextual grounding)	Managed faithfulness gate. Blocks hallucinated responses before returning to caller — the Safety Sandwich.
9	`Latency SLO check`	CloudWatch Alarms + Service Quotas + pybreaker circuit breaker	SLA compliance. Automated failover if agent response exceeds max_latency.
10	`Observability log`	CloudWatch Logs + X-Ray trace + unique traceId	Immutable audit record. X-Ray shows which tool or RAG step caused latency.
11	`State mutation + return`	Bedrock Agent Session State + DynamoDB session store	Multi-turn memory without custom state management. Session context persists across invocations.

3.3 LangGraph vs Bedrock Flows

Feature	LangGraph (E2A Reference)	Amazon Bedrock Flows
Control	Cyclic graphs, exact Python per transition, NFR enforcement per node	Visual DAG canvas, node-based, managed state persistence
Vendor lock-in	None — runs locally, Lambda, or any cloud	AWS-specific — Flow ARN, Bedrock infra required
Testing	Unit-testable per node; async_mode=False for sync test	Requires AWS environment; harder to test locally
Use when	Complex NFR enforcement, multi-cloud portability required	Rapid AWS integration, visual audit, deep native AWS service triggers

3.4 AWS E2A-Compliant Code — RefundAgent

src/agents/refund_agent_aws.py

```
# E2A-compliant AWS RefundAgent. Bedrock is reached through the bound BedrockLLMService
(config['llm_service']).
# The agent contains no model code. All NFR enforcement inherited from BaseAgent /
BaseLLMAgentService.
from src.e2a_base import BaseAgent, resolve_config, GovernanceViolationError
import logging
from src.e2a_base import ConfigKey, register_config_key
register_config_key(ConfigKey('refund_limit', 'REFUND_LIMIT', 500.0, float, 'Refund domain',
'RefundAgent._apply_policy()'))   # project key: src/settings.py, central registry

class RefundAgent(BaseAgent):
    agent_name = 'RefundAgent'   # governance key

    # No _validate_state on the agent. Input validation runs before the agent is built:
    # _validate_refund_agent on the BaseValidationService subclass (Playbook Section 2.2)
    # checks: 'query' in state; 'order_id' and 'refund_amount' in state['metadata'].

    def _apply_policy(self, state, config=None, **kwargs) -> None:
        limit = float(resolve_config('refund_limit', config))
        if float(state.get('metadata',{}).get('refund_amount',0)) > limit:
            raise GovernanceViolationError(f'Refund exceeds auto-limit {limit}')

    def _build_messages(self, state, config=None, **kwargs) -> list:
        m = state.get('metadata', {})
        return [{'role':'user','content':[{'text':
            f"Process refund for Order {m.get('order_id')} ",
            f"Amount ${m.get('refund_amount')}. {state['query']}"}]}]

    def _evaluate_output(self, response, state, config=None, **kwargs) -> float:
        text = str(response).lower()
        return 0.95 if len(text) > 20 and 'not sure' not in text else 0.4

    def _fallback(self, state, config=None, **kwargs) -> str:
        return 'Routed to human supervisor for review.'

    def _handle_error(self, error, state, config=None, **kwargs):
        logging.error(f'RefundAgent: {error}')
        state.setdefault('errors',[]).append(str(error))
        return state

# Invocation:
agent = RefundAgent(config={'llm_service': bedrock_svc, 'model_id':'anthropic.claude-3-5-sonnet-
20241022-v2:0',
    'approved_agents':['RefundAgent'],'refund_limit':1000.0}, state=state)
result = await agent.run(state, agent.config)
```

4. Google Cloud Platform (GCP)

Vertex AI · Gemini · Agent ADK · AlloyDB AI · Cloud Trace

Note on GCP Service Timeline
Services marked * are announced/projected as of May 2026. Verify current GA status before implementation. LangGraph on GCP is the stable E2A reference; ADK is the GCP-native alternative.

4.1 GCP AI Ecosystem — 6-Pillar Breakdown

E2A Pillar	Managed Service	E2A Component Mapped
1. Responsible AI	Cloud IAM, Model Armor*, VPC Service Controls, Gemini Guardrails*, DLP Sensitive Data Protection, Agent Gateway*	BaseGovernanceFramework._apply_policy() → Model Armor inline injection check; Agent Gateway org-level policy
2. Data Management	Vertex AI Vector Search, AlloyDB AI*, BigQuery Vector Search, Vertex AI Search, Dataplex, Spanner Graph	BaseRAGPipeline._search_index(query_text, query_vector) → Vertex Vector Search (hybrid with sparse + dense); _embed_text() → textembedding-gecko; _index_document() → AlloyDB AI*
3. Model Hub	Gemini 2.5 Pro/Flash, Gemma 3, Claude 4 via Model Garden, Llama 4 via Model Garden, Imagen 3	VertexLLMService._dispatch_upstream_call() → Vertex AI Prediction API; Gemini Flash for cost-sensitive agents. Vertex quotas (QPM/TPM) → max_rpm / max_tpm
4. Orchestration	Agent Development Kit (ADK)*, Agent Studio* (no-code), LangChain on Vertex, MCP native*, A2A Protocol*	BaseWorkflow._build_workflow() → LangGraph StateGraph or ADK AgenticSystem; _get_agent() → agent registry
5. Observability	Cloud Logging, Cloud Trace, OpenTelemetry, Agent Observability* (reasoning trace), Agent Inbox* (HITL)	BaseObservability._collect_traces() → Cloud Trace; _collect_metrics() → Cloud Monitoring; _enforce_slo() → Alert Policies
6. GenAI Ops	Agent Evaluation Service*, Vertex AI Custom Training, LoRA fine-tuning, Cloud Build, Terraform, Agent Registry*	BasePipeline._run_rag_eval() → Agent Evaluation Service*; _build_artifacts() → Cloud Build; _deploy_service() → Cloud Run

4.2 E2A to GCP — 11-Step Execution Mapping

#	E2A Step	Managed Tool Equivalent	Enterprise Implication
1	`BaseValidationService.validate() (before run())`	Vertex AI Agent Builder input schema + Apigee API Gateway	Enforces query schema at GCP API edge. Agent Identity* means each agent has its own credentials.
2	`Governance check`	Cloud IAM + Agent Identity* + VPC Service Controls	Zero-trust perimeter per agent. VPC Service Controls prevent data exfiltration even on credential compromise.
3	`_apply_policy`	Model Armor* (injection detection) + Agent Gateway* (policy)	Model Armor scans input before reaching Gemini. Agent Gateway enforces org topic deny-lists across all agents centrally.
4	`__ensure_idempotency`	Spanner Mutations (IF NOT EXISTS) + ADK session_id	One winner per key in globally consistent Spanner (conditional insert), so a retried write is applied once. ADK session lifecycle provides the idempotency key automatically.
5	`_build_messages`	Knowledge Catalog* grounding + Vertex AI Prompt Templates	Unified context across Cloud Storage, BigQuery, SaaS. Prompt templates versioned in Vertex without code deployments.
6	`__track_tokens`	Vertex AI token usage metrics + Cloud Monitoring cost dashboards	Gemini Flash vs Pro cost tracked per agent. Budget alerts trigger Cloud Scheduler to swap model tier automatically.
7	`LLM service — _dispatch_upstream_call`	VertexLLMService → Vertex AI Prediction API — Gemini	Flash for Router/Critic (cost); Pro for Knowledge/Finance (quality) via

		2.5 Pro/Flash unified endpoint	model_id. Quota, concurrency and DLP inherited from BaseLLMAgentService.
8	`_evaluate_output`	Agent Evaluation Service* + Vertex AI Evaluation (groundedness)	Managed RAGAS-equivalent with side-by-side golden dataset comparison. Blocks promotion on faithfulness drop.
9	`Latency SLO check`	Cloud Monitoring Alert Policies + Agent Anomaly Detection*	Anomaly detection identifies regressions. Alert Policies trigger PagerDuty or Agent Inbox* on SLO breach.
10	`Observability log`	Unified Trace Viewer (Cloud Trace + Logging) + OpenTelemetry	End-to-end reasoning trace: tool call latency, RAG retrieval time, model call duration on one timeline.
11	`State mutation`	Vertex AI Managed Sessions (ADK) or Firestore	ADK auto-persists session state; Firestore for custom, queryable session storage with long-term retention.

4.3 LangGraph vs GCP ADK

Feature	LangGraph (E2A Reference)	GCP Agent Development Kit (ADK)
Mental model	State machines — nodes and explicit edges	Hierarchical teams — supervisor + worker agents
State mgmt	AgentState TypedDict with explicit checkpointing	Vertex AI Managed Sessions — zero database management
HITL	interrupt_before/after on specific graph nodes	Agent Inbox* — built-in console for human approval
Use when	Multi-cloud portability, complex cyclic loops, full NFR control	Full GCP stack, Gemini multimodal, rapid deployment with zero infra management

4.4 GCP E2A-Compliant Code — RefundAgent

src/agents/refund_agent_gcp.py

```
# E2A-compliant GCP RefundAgent. Vertex AI is reached through the bound VertexLLMService.
from src.e2a_base import BaseAgent, resolve_config, GovernanceViolationError
import logging
from src.e2a_base import ConfigKey, register_config_key
register_config_key(ConfigKey('refund_limit', 'REFUND_LIMIT', 500.0, float, 'Refund domain',
'RefundAgent._apply_policy()'))   # project key: src/settings.py, central registry

class RefundAgentGCP(BaseAgent):
    agent_name = 'RefundAgent'

    # No _validate_state on the agent: _validate_refund_agent on the BaseValidationService subclass
    # checks 'query', 'order_id', 'amount' before this agent runs (Playbook Section 2.2)

    def _apply_policy(self, state, config=None, **kwargs) -> None:
        limit = float(resolve_config('refund_limit', config))
        if float(state.get('amount', 0)) > limit:
            raise GovernanceViolationError(f'Refund exceeds limit {limit}')

    def _build_messages(self, state, config=None, **kwargs) -> list:
        # Gemini API message format
        return [{'role':'user','parts':[{'text':
            f"Refund: Order {state['order_id']} ",
            f"${state['amount']}. {state['query']}"}]}]
```

```
    def _evaluate_output(self, response, state, config=None, **kwargs) -> float:
        return 1.0 if len(str(response)) > 20 else 0.4

    def _fallback(self, state, config=None, **kwargs) -> str:
        return 'Routed to Agent Inbox for human review.'

    def _handle_error(self, error, state, config=None, **kwargs):
        logging.error(f'GCP RefundAgent: {error}')
        state.setdefault('errors',[]).append(str(error))
        return state

# Invocation — Gemini 2.5 Pro via the bound service:
agent = RefundAgentGCP(config={'llm_service': vertex_svc, 'model_id':'gemini-2.5-pro',
    'approved_agents':['RefundAgent'],'refund_limit':1000.0},state=state)
result = await agent.run(state, agent.config)
```

5. Microsoft Azure

Azure AI Foundry · Azure OpenAI · AI Search · Azure Monitor

5.1 Azure AI Ecosystem — 6-Pillar Breakdown

E2A Pillar	Managed Service	E2A Component Mapped
1. Responsible AI	Azure RBAC + Managed Identity, Azure AI Content Safety, Azure Key Vault, Microsoft Purview, Entra ID	BaseGovernanceFramework._apply_policy() → AI Content Safety (harm categories, jailbreak detection); Purview for data classification
2. Data Management	Azure AI Search (semantic + hybrid), Azure CosmosDB vector, Azure Cache for Redis, Azure Blob Storage, OneLake	BaseRAGPipeline._search_index(query_text, query_vector) → AI Search hybrid retrieval; _embed_text() → Azure OpenAI Embedding (text-embedding-3-large)
3. Model Hub	Azure OpenAI (GPT-4o, o1, o3), Azure AI Model Catalog (Llama 4, Mistral, Phi-4), Azure AI Foundry fine-tuning	AzureOpenAILLMService._dispatch_upstream_call() → Azure OpenAI Chat Completions or AI Foundry endpoint. Deployment TPM/RPM quotas → max_tpm / max_rpm
4. Orchestration	Semantic Kernel, Azure AI Agent Service, Azure Prompt Flow, Foundry Agent, AutoGen	BaseWorkflow._build_workflow() → LangGraph or Semantic Kernel pipeline; compare with Azure AI Agent Service §5.3
5. Observability	Azure Monitor + Log Analytics, Application Insights, Azure AI Tracing, OpenTelemetry, Promptflow traces	BaseObservability._collect_metrics() → Azure Monitor custom metrics; _collect_traces() → Application Insights; _enforce_slo() → Alerts
6. GenAI Ops	Azure AI Evaluation, Azure DevOps Pipelines, GitHub Actions + OIDC, AKS, Azure Container Registry, Terraform	BasePipeline._run_rag_eval() → Azure AI Evaluation; _build_artifacts() → ACR push; _deploy_service() → AKS rolling

5.2 E2A to Azure — 11-Step Execution Mapping

#	E2A Step	Managed Tool Equivalent	Enterprise Implication
1	`BaseValidationService.validate()` (before `run()`)	Azure API Management (APIM) schema validation + Managed Identity	Input contract enforced at APIM edge. Managed Identity ensures agent identity without secret management.

2	`Governance check`	Azure RBAC + Entra ID Conditional Access	Enterprise SSO integration. Conditional Access enforces MFA and device compliance before agent invocation.
3	`_apply_policy`	Azure AI Content Safety (pre-call) + Foundry policy YAML	Content Safety detects hate speech, self-harm, violence before LLM sees input. Policy YAML defines custom harm categories.
4	`__ensure_idempotency`	Azure CosmosDB ETag + idempotency-key header	ETag-based optimistic concurrency prevents duplicate writes. Idempotency-key header standard for Azure Function triggers.
5	`_build_messages`	Azure Prompt Flow prompt templates + AI Foundry versioned prompts	Prompt versioning in Foundry. Prompt Flow enables visual editing without deployment cycles.
6	`__track_tokens`	Azure Monitor custom metrics + Azure Cost Management API	Token usage tracked per deployment, per user, per skill. Cost Management API provides real-time budget alerts.
7	`LLM service — _dispatch_upstream_call`	AzureOpenAILLMService → Azure OpenAI Chat Completions API (GPT-4o/o1/o3/Phi-4)	Single endpoint across OpenAI models. Swap GPT-4o for o3 or Phi-4 via config['model_id'] only. Quota, concurrency and DLP inherited from BaseLLMAgentService.
8	`_evaluate_output`	Azure AI Evaluation (groundedness/coherence/fluency) + Content Safety post-call	Managed evaluation metrics. Content Safety post-generation catch catches policy violations not present in input.
9	`Latency SLO check`	Azure Monitor Alert Rules + Application Insights Smart Detection	Smart Detection identifies latency anomalies automatically. Alert rules trigger Logic App remediation workflows.
10	`Observability log`	Application Insights distributed tracing + Log Analytics workspace	Full end-to-end trace with Promptflow integration. Agent reasoning steps visible in Azure portal.
11	`State mutation`	Azure AI Agent Service Thread Store or CosmosDB	Agent Service manages thread state automatically. CosmosDB for custom, globally distributed session storage.

5.3 LangGraph vs Azure AI Agent Service vs Semantic Kernel

Feature	LangGraph (E2A)	Azure AI Agent Service	Semantic Kernel
Abstraction	Graph nodes + edges	Threads + Assistants API	Plugin + pipeline patterns
State	AgentState TypedDict	Thread Store (managed)	KernelArguments bag
Multi-agent	Supervisor + subgraph pattern	Agent-to-Agent natively	Kernel handoff pattern
Use when	Cross-cloud, full NFR control	Azure-native, HITL approval, file handling	Enterprise .NET stack, C# preference

5.4 Azure E2A-Compliant Code — RefundAgent

src/agents/refund_agent_azure.py

```
# E2A-compliant Azure RefundAgent. Azure OpenAI is reached through the bound AzureOpenAILLMService.
# Corrections: _evaluate_output, _fallback, _handle_error (PROTECTED, single underscore).
from src.e2a_base import BaseAgent, resolve_config, GovernanceViolationError
import logging
from src.e2a_base import ConfigKey, register_config_key
register_config_key(ConfigKey('refund_limit', 'REFUND_LIMIT', 500.0, float, 'Refund domain',
'RefundAgent._apply_policy()'))   # project key: src/settings.py, central registry

class RefundAgentAzure(BaseAgent):
    agent_name = 'RefundAgent'

    # No _validate_state on the agent: _validate_refund_agent on the BaseValidationService subclass
    # checks 'query', 'order_id', 'amount' before this agent runs (Playbook Section 2.2)

    def _apply_policy(self, state, config=None, **kwargs) -> None:
        # Azure AI Content Safety check done at infra level
        # Code-level: enforce business refund limit
        limit = float(resolve_config('refund_limit', config))
        if float(state.get('amount', 0)) > limit:
            raise GovernanceViolationError(f'Refund exceeds limit {limit}')

    def _build_messages(self, state, config=None, **kwargs) -> list:
        # Azure OpenAI Chat Completions message format
        return [
            {'role':'system','content':'You are a refund specialist.'},
            {'role':'user','content':
             f"Order {state['order_id']}, ${state['amount']}: {state['query']}"}
        ]

    def _evaluate_output(self, response, state, config=None, **kwargs) -> float:
        # PROTECTED — single underscore (corrected from evaluate_output)
        return 0.95 if len(str(response)) > 20 else 0.4

    def _fallback(self, state, config=None, **kwargs) -> str:
        # PROTECTED — single underscore (corrected from fallback)
        return 'Escalated to Azure AI Agent Service thread for review.'

    def _handle_error(self, error, state, config=None, **kwargs):
        # PROTECTED — single underscore (corrected from handle_error)
        logging.error(f'Azure RefundAgent: {error}')
        state.setdefault('errors',[]).append(str(error))
        return state

# Invocation — Azure OpenAI GPT-4o:
agent = RefundAgentAzure(
    config={'llm_service': azure_svc, 'model_id':'gpt-4o',
            'azure_endpoint': resolve_config('azure_openai_endpoint'),   # registered in
src/settings.py as AZURE_OPENAI_ENDPOINT
            'approved_agents':['RefundAgent'],'refund_limit':1000.0},
    state=state)
result = await agent.run(state, agent.config)
```

6. Meta Llama

Llama 4 · LangGraph · Standalone · Bedrock / Together AI / Groq

6.1 Meta Llama Ecosystem — 6-Pillar Breakdown

E2A Pillar	Managed Service	E2A Component Mapped
1. Responsible AI	Llama Guard 4 (content moderation), Meta Prompt Guard (injection detection), PrivacyMind (PII filtering), Cybersec Llama (threat detection)	BaseGovernanceFramework._apply_policy() → Llama Guard 4 inline moderation; Meta Prompt Guard injection scan on every _build_messages() output
2. Data Management	ChromaDB / Pinecone (open-source vector), Weaviate, FAISS, pgvector (self-hosted or cloud), Llama-based embedding models	BaseRAGPipeline._search_index(query_text, query_vector) → FAISS or ChromaDB vector leg fused with a local BM25 index; _embed_text() → Llama 4 embedding or nomic-embed-text; _index_document() → local or S3-backed
3. Model Hub	Llama 4 Scout (17B MoE active), Llama 4 Maverick (17B MoE), Llama 4 Behemoth (2T params); via Bedrock, Together AI, Groq, or standalone ollama	BedrockLLMService (managed 'meta.llama4-*') or OllamaLLMService (standalone endpoint) — chosen by which service is bound, not by model_id prefix. vLLM max concurrent sequences → pod_concurrency
4. Orchestration	LangGraph (primary E2A reference), Llama Stack API (emerging standard), LlamaIndex Workflows, LlamaAgents multi-agent	BaseWorkflow._build_workflow() → LangGraph StateGraph; Llama Stack API as emerging alternative for on-prem deployments
5. Observability	Prometheus + Grafana (standalone), OpenTelemetry, LangSmith (LangGraph), Weights & Biases for model evaluation	BaseObservability._collect_metrics() → Prometheus PushGateway; _collect_traces() → OpenTelemetry OTLP; _enforce_slo() → Grafana Alerts
6. GenAI Ops	Unsloth (LoRA fine-tuning), vLLM (serving), Ollama (dev/test), TorchTune, GitHub Actions, Docker/K8s self-hosted	BasePipeline._run_rag_eval() → custom RAGAS harness; _build_artifacts() → Docker + vLLM image; _deploy_service() → K8s or ECS

6.2 Standalone vs Cloud Deployment

Dimension	Standalone / On-Prem (Ollama / vLLM)	Cloud-Hosted (Bedrock / Together AI / Groq)
Data sovereignty	✓ Full control — data never leaves your infra	Provider terms apply; region-based data residency
Cost model	CapEx: GPU hardware; OpEx: power, ops team	OpEx only: per-token pricing; no hardware management
Latency	Local inference — lowest latency for edge/on-prem	Network RTT applies; Groq LPU gives ~10x throughput
Scale-out	Manual Kubernetes scaling; vLLM + tensor parallelism	Auto-scaling managed; Together AI / Groq scale to traffic
E2A config	`config={'llm service': ollama svc, 'model_id':'llama4-scout-17b'} # endpoint set on the service`	`config={'llm service': bedrock svc, 'model_id':'meta.llama4-maverick-17b-instruct-v1:0'}`

6.3 LangGraph vs Llama Stack API

Feature	LangGraph (E2A Reference)	Llama Stack API
Status	Production-stable, widely adopted	Emerging standard (Meta open specification)
Strength	Full NFR enforcement, cyclic reasoning, multi-cloud	Unified API across Llama providers; swap Bedrock/Together/Groq via config

State	AgentState TypedDict — explicit, typed	Session-based — Llama Stack manages context
Use when	Production multi-cloud E2A deployment	Llama-centric, on-prem + Bedrock hybrid, emerging Llama ecosystem

6.4 Meta Llama E2A-Compliant Code — RefundAgent

src/agents/refund_agent_llama.py

```
# E2A-compliant Llama RefundAgent.
# E2A-compliant Llama RefundAgent.
# Works with Llama 4 via Bedrock or standalone (ollama) — the bound LLM service decides.
from src.e2a_base import BaseAgent, resolve_config, GovernanceViolationError
import logging
from src.e2a_base import ConfigKey, register_config_key
register_config_key(ConfigKey('refund_limit', 'REFUND_LIMIT', 500.0, float, 'Refund domain',
'RefundAgent._apply_policy()'))   # project key: src/settings.py, central registry
from datetime import datetime, timezone

class RefundAgentLlama(BaseAgent):
    agent_name = 'RefundAgent'

    # No _validate_state on the agent: _validate_refund_agent on the BaseValidationService subclass
    # checks 'query', 'order_id', 'amount' before this agent runs (Playbook Section 2.2)

    def _apply_policy(self, state, config=None, **kwargs) -> None:
        # Llama Guard 4 scan injected here at infra level
        # Code-level: enforce business rule
        limit = float(resolve_config('refund_limit', config))
        if float(state.get('amount', 0)) > limit:
            raise GovernanceViolationError(f'Refund exceeds limit {limit}')

    def _build_messages(self, state, config=None, **kwargs) -> list:
        # Llama-compatible chat format
        return [
            {'role':'system','content':'You are a refund specialist. Be concise.'},
            {'role':'user','content':
             f"Refund request: Order {state['order_id']}, "
             f"Amount ${state['amount']}. {state['query']}"}
        ]

    def _evaluate_output(self, response, state, config=None, **kwargs) -> float:
        # PROTECTED — single underscore (corrected)
        return 0.95 if len(str(response)) > 20 else 0.4

    def _fallback(self, state, config=None, **kwargs) -> str:
        # PROTECTED — single underscore (corrected)
        return 'Routed to human agent for manual review.'

    def _handle_error(self, error, state, config=None, **kwargs):
        # PROTECTED — single underscore (corrected)
        # datetime.utcnow() corrects os.getlogin() timestamp bug
        ts = datetime.now(timezone.utc).isoformat()
        logging.error(f'[{ts}] Llama RefundAgent: {error}')
        state.setdefault('errors',[]).append(str(error))
        return state

# Invocation — Cloud (Bedrock):
agent = RefundAgentLlama(
    config={'llm_service': bedrock_svc, 'model_id':'meta.llama4-maverick-17b-instruct-v1:0',
            'approved_agents':['RefundAgent'],'refund_limit':1000.0},
    state=state)
result = await agent.run(state, agent.config)
```

```
# Invocation — Standalone (ollama):
agent = RefundAgentLlama(
    config={'llm_service': ollama_svc, 'model_id':'llama4-scout-17b',
            'endpoint':'http://localhost:11434/api/chat',
            'approved_agents':['RefundAgent'],'refund_limit':1000.0},
    state=state)
result = await agent.run(state, agent.config)
```

7. Cross-Cloud Comparison

6-Pillar · Orchestration · Anti-Lock-In

7.1 Six-Pillar Side-by-Side

Pillar	AWS	GCP	Azure	Meta Llama
Responsible AI	Bedrock Guardrails (denied topics, PII, grounding)	Model Armor* Agent Gateway*	AI Content Safety Purview DLP	Llama Guard 4 Meta Prompt Guard
Data / RAG	OpenSearch hybrid Bedrock Knowledge Bases	Vertex Vector Search AlloyDB AI*	AI Search hybrid CosmosDB vector	FAISS / ChromaDB pgvector self-hosted
Model Hub	Nova, Claude 3.5/4 Llama 4, Mistral via KB	Gemini 2.5 Pro/Flash Claude/Llama via Garden	GPT-4o, o1/o3, Phi-4 Llama 4 via Catalog	Llama 4 Scout/Maverick Behemoth (coming)
Orchestration	Bedrock Agents Bedrock Flows	ADK*, Agent Studio* LangGraph on Vertex	AI Agent Service Semantic Kernel	LangGraph (primary) Llama Stack API
Observability	CloudWatch + X-Ray CloudTrail audit	Cloud Trace + Logging Agent Observability*	App Insights Azure Monitor	Prometheus + Grafana OpenTelemetry OTLP
GenAI Ops	Bedrock Evaluation SageMaker MLflow	Agent Evaluation* Vertex Training	Azure AI Evaluation DevOps Pipelines	RAGAS custom vLLM + Docker
Model quota	Per-model, per-region RPM/TPM limits; throttling error	Per-model quota or provisioned throughput	Per-deployment TPM/RPM; provisioned units	Set by the serving stack you run
Compute scale-out	ECS service auto scaling	Cloud Run concurrency and max instances	Container Apps replicas with KEDA	Serving-stack replicas (e.g. vLLM)
Conditional writes	DynamoDB condition expressions (single item)	Firestore preconditions and transactions	Cosmos DB ETag If-Match; batch within one partition	Host store's own version check (e.g. PostgreSQL)
Managed locks / leases	Versioned lease item; no native fencing token	Versioned document or object generation precondition	Blob lease or versioned item	Host store; adapter supplies the fencing token
Queue partitioning / backpressure	SQS FIFO message group; visibility timeout; DLQ redrive	Pub/Sub ordering key; ack deadline; dead-letter topic	Service Bus sessions; peek-lock; max delivery count	Host-chosen broker; same contract
Concurrency observability	CloudWatch queue depth and oldest-message age	Cloud Monitoring oldest unacked message age	Azure Monitor; Service Bus has no native oldest-message-age metric	Prometheus + OpenTelemetry

Third-Party Disclaimer: *This reference includes third-party partner services (e.g., Lakera, Pinecone, LangGraph) that are part of the broader cloud ecosystem but are not native managed services of the respective Cloud Service Providers (CSPs). Users should verify separate licensing and compliance terms for these entities.*

7.2 Orchestration Framework Comparison

Feature	LangGraph	Bedrock Flows	GCP ADK*	Semantic Kernel
E2A alignment	✓ Native — reference impl	Partial — AWS-specific	Partial — GCP-specific	Partial — Azure/.NET focus
State model	AgentState TypedDict	Managed AWS state	ADK managed sessions	KernelArguments
Multi-cloud	✓ Yes	✗ AWS only	✗ GCP only	✓ Azure/local
Cyclic loops	✓ Native	Limited — DAG only	✓ Supported	Plugin chains
NFR enforcement	✓ Full per-node control	Guardrails at flow level	Agent-level policies	Filter middleware
Best for	All E2A deployments	Rapid AWS-native MVPs	GCP Gemini-native	Enterprise .NET orgs

7.3 Anti-Lock-In Strategy

The E2A Framework's abstraction provides both architectural and economic lock-in protection. The key mechanism is that the two transport hooks of a BaseLLMAgentService subclass are the only place where a model vendor SDK is invoked. Every other method is either abstract (subclass provides domain logic) or uses standard Python libraries.

E2A Component	What changes when switching vendors	What stays identical
BaseAgent subclasses	Nothing — the agent never sees the provider; the bound config['llm_service'] changes	All PROTECTED method implementations (_build_messages, _apply_policy, _evaluate_output, _fallback, _handle_error)
BaseRAGPipeline subclasses	_embed_text() SDK call + _search_index() client (one method each)	retrieve() NFR enforcement, reranking, faithfulness gate, all config resolution
BaseToolService subclasses	get_endpoint() URL prefix (one line in config)	execute() retry/timeout/auth, _validate_input(), _build_payload()
BaseWorkflow subclasses	_get_agent() registry (if agent names change)	execute() governance check, entire graph topology, routing logic
BaseLLMAgentService subclass	_dispatch_upstream_call() and _dispatch_upstream_stream() — one subclass per provider	Pod semaphore, RPM/TPM bucket, context budgeting, KV/semantic caching hooks, DLP, usage accounting, failure semantics, every line of e2a_base.py

Runtime Vendor Arbitrage

Because the SAME subclass runs on any provider via config, you can implement cost optimisation without code changes:

```
# Route cheaply during off-peak, quality model during business hours
import datetime
hour = datetime.datetime.utcnow().hour
model = 'meta.llama4-scout' if hour < 8 or hour > 20 else 'anthropic.claude-3-5-sonnet'
result = agent.run(state, {'llm_service': bedrock_svc, 'model_id': model, **base_config})   # same provider
# Across providers: bind a different service — still no agent change:  {'llm_service': azure_svc, 'model_id': 'gpt-4o'}
```

No agent subclass is modified. No redeployment. Zero code changes. This is the E2A Framework's economic value proposition formalised as a pattern.

7.4 Concurrency Mechanisms — Application Invariant vs Provider Mechanism

The E2A framework states each concurrency guarantee as an application invariant — one winner per idempotency key, one writer per version, one leader per lease — and leaves the mechanism to the provider adapter. The rows below name a mechanism that can carry each invariant on each cloud. They are not a claim that the services are equivalent: the last column states where behaviour differs, and the adapter and its tests, not the service name, are what establish the invariant.

Application invariant	AWS mechanism	GCP mechanism	Azure mechanism	Where behaviour differs
One winner per idempotency key	DynamoDB conditional put	Firestore create-if-absent	Cosmos DB create item	Scope is one item or partition. Expiry must be an explicit field: TTL deletion is delayed on AWS and GCP, while Cosmos DB hides expired items immediately. TTL is garbage collection only.
One writer per version	Condition expression on a version attribute	Update-time precondition or transaction	ETag with If-Match	DynamoDB conditional writes are single-item; Cosmos DB batches are limited to one logical partition. An invariant spanning items needs a transaction or confinement to one item.
One leader, with a fencing token	Versioned lease item	Versioned document	Blob lease or versioned item	No provider offers a native monotonic fencing token. The adapter supplies one and the receiving store must reject an older token.
Redelivery and dead-lettering	SQS redrive policy	Pub/Sub dead-letter topic	Service Bus max delivery count	At-least-once on all three; duplicate-suppression windows differ and are not relied on.
Ordered processing	SQS FIFO message group	Pub/Sub ordering key	Service Bus session	A poison message blocks its group on SQS FIFO until it reaches the DLQ; Pub/Sub redelivers the key tail; Service Bus dead-letters and moves on.
Bounded model usage	Per-model RPM/TPM	Per-model quota or provisioned throughput	Per-deployment TPM/RPM	The framework quota is an estimate refunded to actual usage. How Azure OpenAI counts max_tokens against TPM is not settled here and no rule is assumed.
Graceful drain	ECS 30 s default, up to 120 s	Cloud Run about 10 s	Container Apps 30 s	drain_timeout_seconds must be shorter than the platform grace period.
Queue age alarm	Oldest-message age	Oldest unacked message age	No native metric; derive from the	Alarm on age where it exists, on depth and

			enqueue time of a peeked message	derived age where it does not.

Vendor behaviour above reflects provider documentation reviewed for this revision and should be re-verified against current documentation before it is relied on in a design. The Azure max_tokens quota-accounting rule is deliberately left unresolved.

E2A Architecture Framework — Multi-Cloud AI Ecosystem Reference · github.com/subhamviky/e2a-framework · Subham Gupta

Authorship: Framework implementation, abstract class contracts, and multi-cloud mapping logic are the original work of Subham Gupta.

Trademark: All vendor trademarks (AWS, GCP, Azure, Meta Llama) are the property of their respective owners and are used here for architectural reference only.

Managed Service Disclaimer: Certain ecosystem components referenced (e.g., Pinecone, LangGraph, Lakera, FAISS) are third-party partner technologies and are not native managed services of AWS, Google Cloud, or Microsoft Azure. The E2A Framework provides a standardized way to integrate these third-party tools alongside native cloud services.